

ESP32 → Node.js → MongoDB Atlas

Complete IoT Setup Guide

From Zero to Global API in Simple Steps

Prepared for: Bijoy Kumar Bhowmik (neuro-bijoy)

Techno College of Engineering Agartala (TCEA)

July 2026

PROJECT OVERVIEW

This guide documents the complete process of building an IoT backend system using ESP32 microcontroller, Node.js + Express REST API, and MongoDB Atlas cloud database. The system allows ESP32 to send sensor data to a globally hosted server that stores data permanently in the cloud.

System Architecture

The system follows a three-tier architecture:

Layer	Technology	Purpose
Device Layer	ESP32 Microcontroller	Sends sensor data via HTTP POST
Server Layer	Node.js + Express on Render.com	Receives data and forwards to database
Database Layer	MongoDB Atlas (Cloud)	Stores all data permanently
Client Layer	MIT App Inventor / Browser	Reads and displays data

Data Flow

ESP32 → HTTP POST → Render.com (Node.js API) → MongoDB Atlas
MIT App Inventor → HTTP GET → Render.com API → MongoDB Atlas

PHASE 1 — MONGODB ATLAS SETUP

MongoDB Atlas is the cloud database where all sensor data is stored permanently. It is free up to 512 MB and does not require your laptop to be on.

STEP 1 Create MongoDB Atlas Account

- Go to cloud.mongodb.com and sign up for a free account.
- Create a new Project called 'Project 0'.
- Create a free cluster called 'Cluster0' (choose the free M0 tier).

STEP 2 Create Database User

- Go to Security → Database Access.
- Click 'Add New Database User'.
- Username: `bijoycetcea2027_db_user`
- Password: create a strong password and save it safely.
- Role: Read and Write to any database.

STEP 3 Configure Network Access

- Go to Security → Network Access.
- Click '+ ADD IP ADDRESS'.
- Enter `0.0.0.0/0` to allow connections from all IP addresses.
- This is important because your hotspot IP changes every time.
- Click Confirm.

STEP 4 Get Connection String

- Go to Cluster0 → Connect → Drivers.
- IMPORTANT: Enable the 'Legacy URI String' toggle.
- Copy the connection string that starts with `mongodb://` (NOT `mongodb+srv://`).

The correct Legacy URI format looks like:

```
mongodb://username:password@ac-who6tan-shard-00-00.sqir24c.mongodb.net:27017,  
ac-who6tan-shard-00-01.sqir24c.mongodb.net:27017,  
ac-who6tan-shard-00-02.sqir24c.mongodb.net:27017/ESP32?ssl=true&replicaSet=atlas-11q8y7-shard-  
0&authSource=admin
```

Common Error: If you use `mongodb+srv://` instead of `mongodb://`, you will get this error:

```
querySrv ECONNREFUSED _mongodb._tcp.cluster0.sqir24c.mongodb.net
```

Solution: Always use the Legacy URI starting with `mongodb://`.

PHASE 2 — NODE.JS + EXPRESS API SETUP

The Node.js server acts as a middleware between ESP32 and MongoDB Atlas. ESP32 cannot connect directly to MongoDB, so this API server handles all communication.

STEP 1 Install Node.js

- Download Node.js from nodejs.org.
- Install with default settings.
- Verify by running: `node --version` in terminal.

STEP 2 Create Project Folder

```
mkdir MongoAPI
cd MongoAPI
npm init -y
```

STEP 3 Install Required Packages

```
npm install express mongoose cors dotenv
npm install --save-dev nodemon
```

Package	Purpose
express	Creates the REST API server
mongoose	Connects Node.js to MongoDB Atlas
cors	Allows requests from ESP32 and apps
dotenv	Stores MongoDB password securely
nodemon	Auto-restarts server during development

STEP 4 Create .env File

Create a file named `.env` in the MongoAPI folder. This file stores your MongoDB connection string securely.

```
MONGO_URI=mongodb://bijoyecetcea2027_db_user:YOUR_PASSWORD@ac-who6tan-shard-00-00...
PORT=3000
```

STEP 5 Create .gitignore File

Create a file named `.gitignore` to prevent sensitive files from being uploaded to GitHub.

```
.env
node_modules/
```

STEP 6 Write server.js — Flexible Schema

This `server.js` uses a flexible (Mixed) schema so ESP32 can send ANY JSON data without ever needing to change the server code again.

```
require('dotenv').config();
const express = require('express');
```

```
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());
```

```
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log('Error', err));
```

```
const dataSchema = new mongoose.Schema({
  data: mongoose.Schema.Types.Mixed,
  timestamp: { type: Date, default: Date.now }
});
const Data = mongoose.model('Data', dataSchema);
```

```
app.post('/data', async (req, res) => {
  const newData = new Data({ data: req.body });
  await newData.save();
  res.json({ status: 'saved', data: newData });
});
```

```
app.get('/data', async (req, res) => {
  const allData = await Data.find();
  res.json(allData);
});
```

```
app.listen(process.env.PORT || 3000, () => {
  console.log('Server running on port 3000');
});
```

Key advantage of Mixed schema: ESP32 can send temperature, GPS coordinates, humidity, or any other data — the server saves everything without modification.

PHASE 3 — UPLOAD TO GITHUB

GitHub is used to store your code online. Render.com reads from GitHub to deploy your server automatically.

STEP 1 Install Git

- Download Git from git-scm.com/download/win.
- Install with default settings.
- Verify: `git --version` (should show `git version 2.55.0.windows.2`).

STEP 2 Configure Git Identity

```
git config --global user.email "your@email.com"
git config --global user.name "neuro-bijoy"
```

STEP 3 Create GitHub Repository

- Go to github.com and login.
- Click '+' → 'New repository'.
- Name: `Mongo_API`
- Keep it Public.
- Do NOT tick any checkboxes (no README, no `.gitignore`).
- Click 'Create repository'.

STEP 4 Push Code to GitHub

```
git init
git add .
git commit -m "first commit"
git remote add origin https://github.com/neuro-bijoy/Mongo_API.git
git push -u origin master
```

STEP 5 Update Code in Future

Every time you change `server.js`, run these 3 commands:

```
git add .
git commit -m "describe your change"
git push origin master
```

Render.com automatically detects the change and redeploys within 2-3 minutes.

TIP: You can also edit `server.js` directly on GitHub website by clicking the pencil icon. Render will auto-deploy after every save.

PHASE 4 — DEPLOY TO RENDER.COM

Render.com hosts your Node.js server 24/7 for free. Once deployed, your API works globally even when your laptop is turned off.

STEP 1 Sign Up on Render

- Go to render.com.
- Sign up using your GitHub account.

STEP 2 Create New Web Service

- Click 'New +' → 'Web Service'.
- Click 'Connect a repository'.
- Select `neuro-bijoy/Mongo_API`.
- Click 'Install' to give Render access to GitHub.

STEP 3 Configure the Service

Field	Value
Name	Mongo_API
Language	Node
Branch	master
Build Command	npm install
Start Command	node server.js
Instance Type	Free

STEP 4 Add Environment Variables

Scroll down to Environment Variables section and add:

Key	Value
MONGO_URI	Your full mongodb:// connection string
PORT	3000

STEP 5 Deploy

- Click 'Deploy Web Service'.
- Wait 2-3 minutes for build to complete.
- Status will show 'Live' in green when ready.

Your public API URL:

<https://mongo-api-f7it.onrender.com>

Endpoint	Method	Purpose
/data	POST	ESP32 sends data to MongoDB
/data	GET	Read all stored data in browser

Important Note: Render free plan sleeps after 15 minutes of inactivity. First request takes 50 seconds to wake up. If ESP32 sends data every 5 seconds, the server never sleeps.

PHASE 5 — ESP32 ARDUINO CODE

This code connects ESP32 to your mobile hotspot and sends data to your Render API every 5 seconds.

STEP 1 Basic Hello World Test Code

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "YOUR_HOTSPOT_NAME";
const char* password = "YOUR_HOTSPOT_PASSWORD";
const char* serverURL = "https://mongo-api-f7it.onrender.com/data";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi Connected!");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(serverURL);
    http.addHeader("Content-Type", "application/json");
    String json = "{\"message\": \"Hello World\"}";
    int code = http.POST(json);
    Serial.println("Response: " + String(code));
    http.end();
  }
  delay(5000);
}
```

STEP 2 GPS Data Code

Replace the json line with GPS coordinates:

```
float latitude = 24.8170;
float longitude = 91.2772;
String json = "{\"latitude\": " + String(latitude, 6) +
  ", \"longitude\": " + String(longitude, 6) + "}";
```

STEP 3 Upload Troubleshooting

If you get 'Failed to connect to ESP32: No serial data received' error:

- Click Upload in Arduino IDE.
- When you see 'Connecting.....!', immediately press and hold the BOOT button on ESP32.
- Hold until you see 'Writing at 0x00001000...' then release.

PHASE 6 — VIEWING YOUR DATA

Method 1 — Browser (Easiest)

Open any browser and visit:

<https://mongo-api-f7it.onrender.com/data>

You will see all data stored in MongoDB in JSON format.

Method 2 — MongoDB Atlas Website

- Go to cloud.mongodb.com and login.
- Click 'Browse Collections' on Cluster0.
- Open database: ESP32 → collection: datas.
- All entries are visible with timestamps.

Data Format in MongoDB

Each entry saved by ESP32 looks like this:

```
{
  "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
  "data": {
    "latitude": 24.817,
    "longitude": 91.277
  },
  "timestamp": "2026-07-04T19:00:00.000Z"
}
```

QUICK REFERENCE

Important URLs

Service	URL
Your API (GET data)	https://mongo-api-f7it.onrender.com/data
Render Dashboard	dashboard.render.com
MongoDB Atlas	cloud.mongodb.com
GitHub Repository	github.com/neuro-bijoy/Mongo_API

Free Plan Limits

Service	Limit	Status
Render.com	Unlimited requests, sleeps after 15min inactivity	Free
MongoDB Atlas	512 MB storage (~2.5 million GPS entries)	Free
GitHub	Unlimited public repositories	Free

Git Commands Cheat Sheet

Action	Command
Check git version	<code>git --version</code>
Initialize repo	<code>git init</code>
Stage all changes	<code>git add .</code>
Commit changes	<code>git commit -m "message"</code>
Push to GitHub	<code>git push origin master</code>
Check file contents	<code>Get-Content .env</code>

Common Errors and Fixes

Error	Cause	Fix
querySrv ECONNREFUSED	Using mongodb+srv:// URI	Use Legacy mongodb:// URI
IP not whitelisted	Current IP not in Atlas	Add 0.0.0.0/0 in Network Access
Cannot GET /data	Route not in server.js	Add POST/GET /data routes
Failed to connect ESP32	Upload timing issue	Hold BOOT button during upload
nothing to commit	File not saved	Press Ctrl+S before git add